

SMART CAMPUS RESOURCE MANAGEMENT SYSTEM

Project Report

Student	Akshit Singh
Programme	B.Tech — Computer Science Engineering
Institution	VIT Bhopal University
Technology	Java
Project Type	Console-Based Application

GitHub Repository

<https://github.com/akshitsingh69/SmartCampusSystem>

1. Introduction

The Smart Campus Resource Management System is a Java console application designed to manage students, shared campus resources, bookings, and concurrent booking requests. It applies core Java and Object-Oriented Programming concepts to a realistic campus problem. The application provides a menu-driven interface for adding and searching students, managing resources, creating bookings, viewing records, and generating basic analytics. A multithreading module demonstrates how synchronization can protect shared resource state when multiple users attempt to book the same resource.

2. Problem Statement

Manual or unstructured management of campus resources can cause booking conflicts, difficulty tracking availability, and inconsistent records. The proposed system provides a simple software solution that organizes student and resource information, controls booking operations, reports basic usage statistics, and demonstrates safe handling of concurrent requests.

3. Functional Requirements

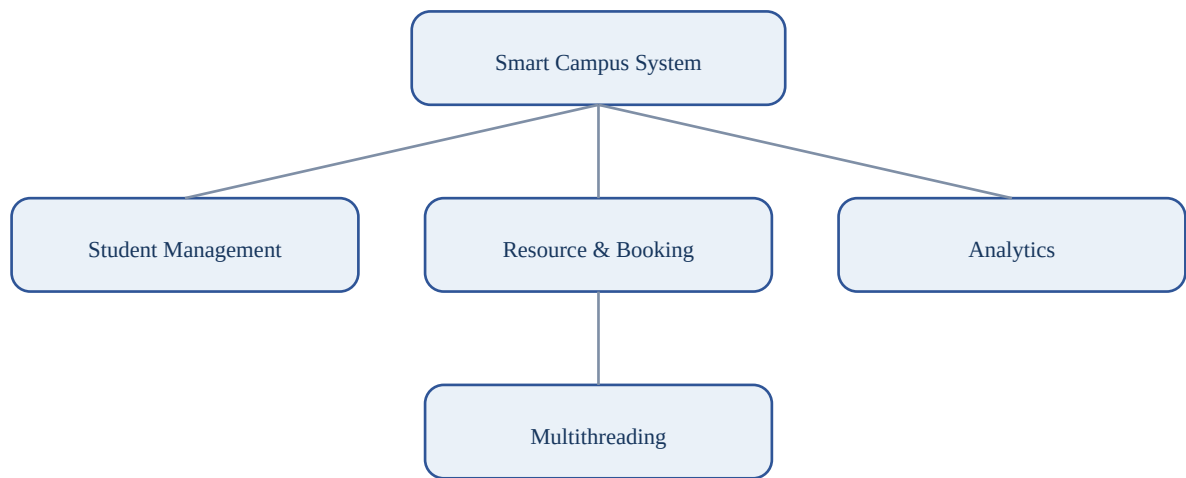
ID	Requirement	Description
FR-01	Student Management	Add, view, search, and sort student records.
FR-02	Resource Management	Add resources and track availability.
FR-03	Booking Management	Book available resources and prevent duplicate booking.
FR-04	Booking Records	Display student, resource, and booking time.
FR-05	Analytics	Display counts and resource availability.
FR-06	Multithreading	Simulate simultaneous booking requests.
FR-07	Validation	Handle invalid input and booking conflicts.

4. Non-Functional Requirements

Requirement	Target
Performance	Quick response for normal academic workloads.
Usability	Simple menu-driven interaction and clear messages.
Reliability	Consistent resource state during booking.
Maintainability	Logical classes, methods, and meaningful naming.
Error Handling	Handle invalid input without abrupt termination.
Scalability	Allow additional users/resources and future storage.
Resource Efficiency	Use lightweight standard Java collections.

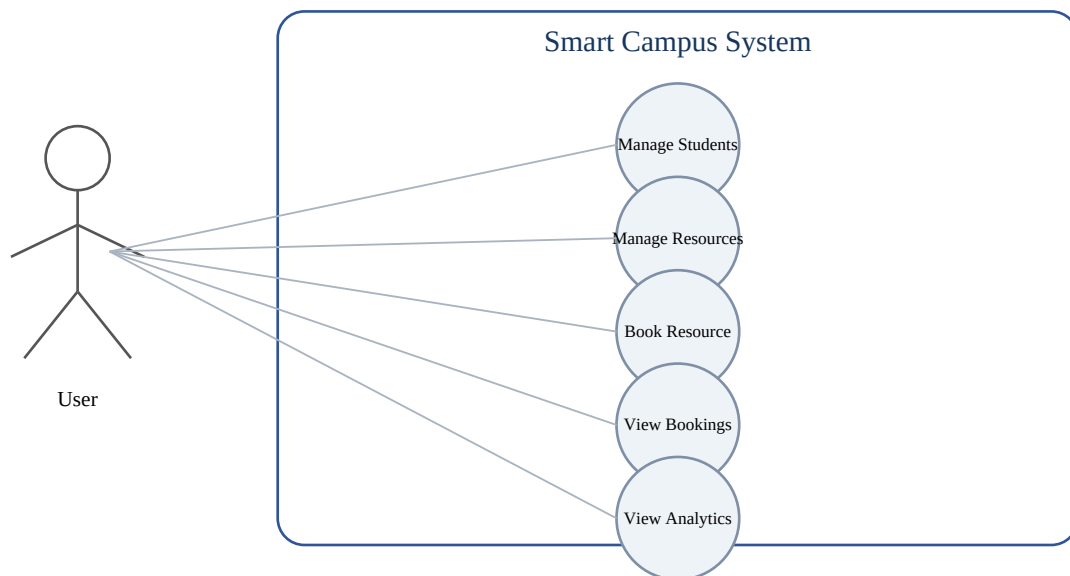
5. System Architecture

The application uses a simple logical architecture. The main method handles console interaction, CampusManager coordinates operations, domain classes represent campus entities, and BookingTask implements concurrent booking behavior.

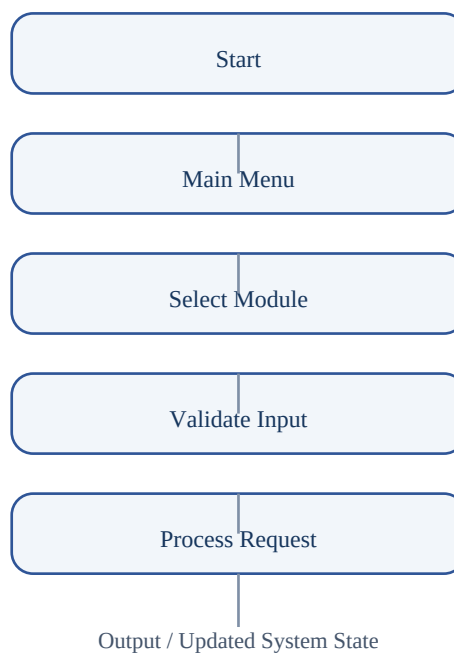


6. Design Diagrams

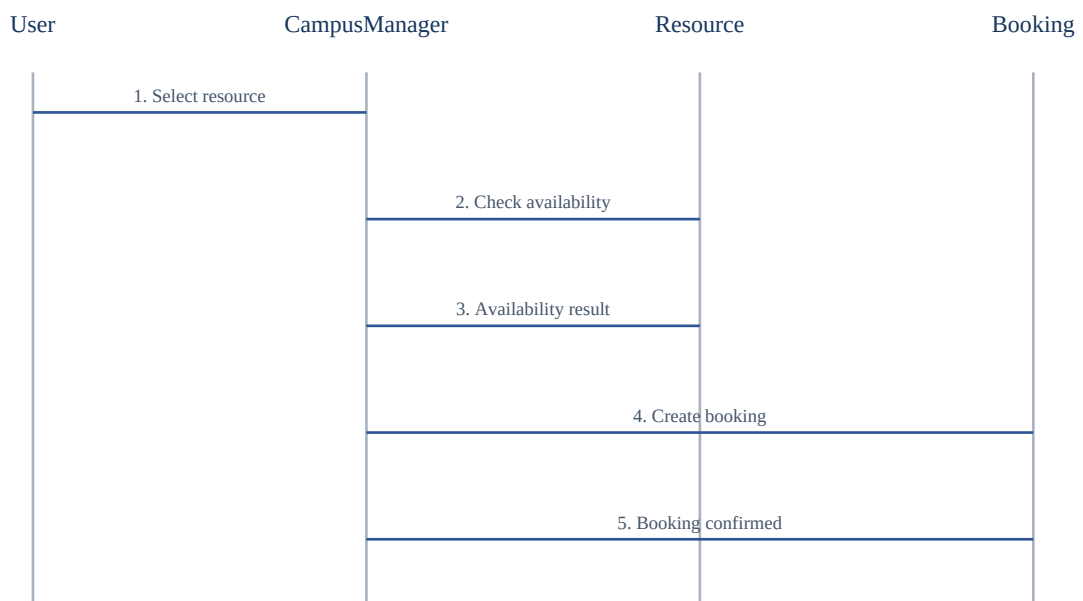
6.1 Use Case Diagram



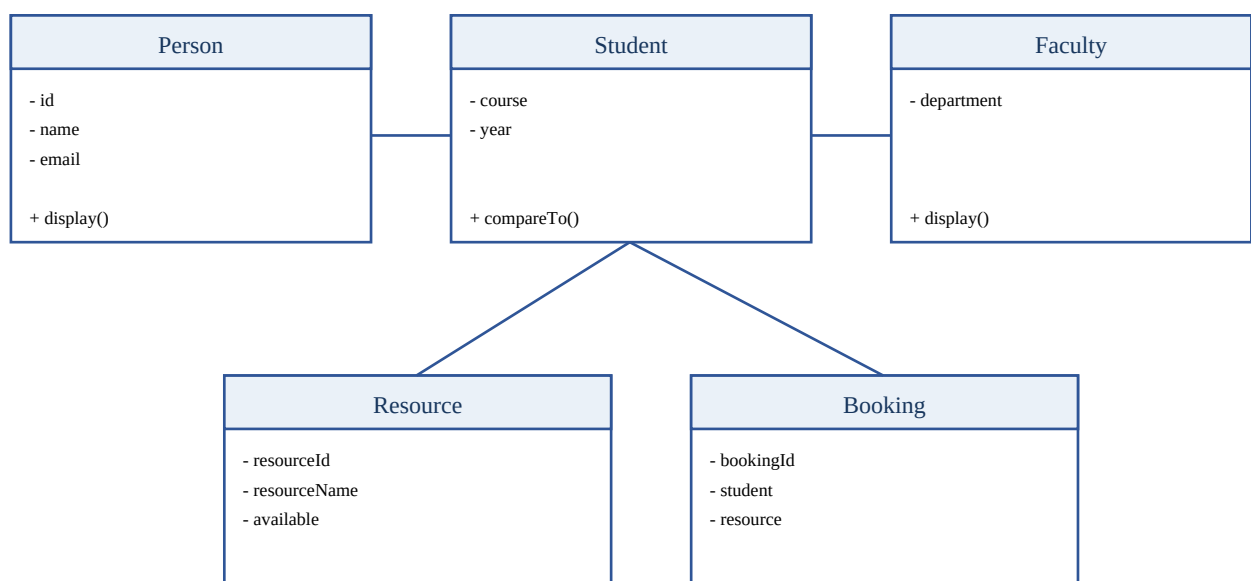
6.2 Workflow Diagram



6.3 Sequence Diagram — Resource Booking



6.4 Class Diagram



6.5 ER Diagram / Storage Design

The current implementation does not use an external database, so a formal ER diagram is not applicable. Data is held in memory using Java collections.

Entity	Fields	Relationship
Student	id, name, email, course, year	A student can make bookings.
Resource	resourceId, resourceName, type, available	A resource can be booked.
Booking	bookingId, student, resource, bookingTime	Connects a student and resource.

7. Design Decisions & Rationale

Decision	Rationale
Single Java source file	Simplifies compilation and demonstrates multiple logical classes in one file.
Abstract Person class	Shares common fields and demonstrates abstraction/inheritance.
Displayable interface	Demonstrates interface-based abstraction.
ArrayList + HashMap	Convenient collection storage and fast student lookup by ID.
Comparable<Student>	Supports alphabetical student sorting.
Custom exception	Makes booking conflicts explicit.
synchronized booking method	Protects shared resource state during concurrent access.
In-memory storage	Keeps the project focused on Java/OOP/multithreading.

8. Implementation Details

8.1 Classes and Responsibilities

Class / Interface	Responsibility
Displayable	Defines display behavior.
Person	Abstract base class for common person information.
Student	Student data and Comparable implementation.
Faculty	Second Person subtype for polymorphism.
Resource	Resource details and availability state.
Booking	Successful booking record.
CampusManager	Coordinates collections, booking, and analytics.
BookingTask	Runnable task for concurrent booking.
Notification	Demonstrates method overloading.
ResourceNotAvailableException	Custom booking-conflict exception.

8.2 OOP Concepts Demonstrated

Concept	Project Example
Encapsulation	Private fields with getters/setters.
Inheritance	Student and Faculty extend Person.
Abstraction	Abstract Person class and Displayable interface.
Polymorphism	Person references point to Student/Faculty objects.

Concept	Project Example
Overriding	display() and getRole() overridden.
Overloading	Notification.send() has multiple signatures.
this / super	Used in constructors.
static / final	Booking counter and project-name constant.

8.3 Multithreading

BookingTask implements Runnable. Two Thread objects can attempt to book the same Resource. The synchronized booking method protects the availability check and state update. Thread.sleep() simulates processing time and join() waits for both threads to finish.

9. Screenshots / Results

The application is console-based. The following result views should be captured from the running program for the final submission if screenshots are required.

Screenshot	Expected evidence
Main Menu	Available system operations.
Student List	Registered students.
Resource List	Resource status.
Successful Booking	Valid booking confirmation.
Booking Conflict	Custom exception when already booked.
Analytics	Aggregate campus counts.
Multithreading Demo	Two threads competing for one resource.

Representative multithreading result:

```
Thread-Student-1 is trying to book Seminar Hall
Booking successful for Akshit Singh
Thread-Student-2 is trying to book Seminar Hall
FAILED: Resource Seminar Hall is already booked.
Both booking threads completed.
```

10. Testing Approach

Testing focuses on functional correctness, validation, booking-state consistency, and concurrent behavior. Manual functional tests can be performed through the menu-driven interface.

Test Case	Action	Expected Result
TC-01	Add valid student	Student added successfully.
TC-02	Search existing ID	Correct details displayed.
TC-03	Search unknown ID	Not found message.

Test Case	Action	Expected Result
TC-04	Add resource	Resource becomes Available.
TC-05	Book available resource	Booking created; resource becomes Booked.
TC-06	Book unavailable resource	Custom exception displayed.
TC-07	Sort students	Students shown alphabetically.
TC-08	Analytics	Correct aggregate values.
TC-09	Concurrent booking	Only one booking succeeds for the shared resource.
TC-10	Invalid menu input	Input handled without crash.

11. Challenges Faced

- Managing shared state: resource availability is shared and must remain consistent during concurrent access.
- Combining concepts: OOP, collections, exceptions, and multithreading were integrated into one understandable application.
- Input validation: console input can produce type errors, so exception handling was included.
- Single-file organization: nested static classes preserve logical modularity while keeping the submission to one source file.
- In-memory storage: data is temporary because no database is used in the current academic version.

12. Learnings & Key Takeaways

The project provides practical experience with Java classes and objects, abstraction, inheritance, polymorphism, interfaces, collections, generics, exception handling, and concurrent programming. The booking simulation demonstrates why synchronization is important when multiple threads operate on shared data.

13. Future Enhancements

- Add MySQL/PostgreSQL database connectivity.
- Develop a JavaFX or web-based graphical interface.
- Add authentication and role-based access.
- Add cancellation, time slots, and booking history.
- Add advanced reports and charts.
- Add automated JUnit tests.
- Split the single source into production-style packages and files.

14. References

1. Oracle Java Documentation — Java Platform, Standard Edition API documentation.
2. Oracle Java Tutorials — Object-Oriented Programming Concepts.
3. Oracle Java Tutorials — Concurrency and Multithreading.

4. Oracle Java Tutorials — Collections Framework.

5. GitHub repository for the project — Smart Campus Resource Management System.

Project Repository: <https://github.com/akshitsingh69/SmartCampusSystem>